Detecting SQL Injection Attacks with NLP: A Comparative Study of Classical and Transformer-Based Models Under Evasion Conditions

Sash Upadhyaya | UC Berkeley School of Information | DATASCI 266 | sashu@berkeley.edu

Abstract

SQL injection (SQLi) is one of the most common and damaging attacks used by malicious actors against web applications that use databases to store and retrieve information. It consistently ranks among OWASP Top 10 security exploits (OWASP Foundation, 2021). In this paper, we compare four NLP models to identify and prevent SQL injection. We use machine learning, deep learning, and NLP transformer-based models to classify SQL queries as benign or malicious. The models include Logistic Regression with TF-IDF features, a Convolutional Neural Network (CNN), DistilBERT, and BERT. We evaluate each model using three test sets. The first contains a mix of SQL injection and normal queries. The other two test sets contain obfuscated SQL injection designed to mimic real-world evasion techniques.

The results show that transformer-based models, especially DistilBERT and BERT, remain highly accurate even under different obfuscation techniques. Machine and deep learning models show a significant drop in SQL injection detection accuracy when obfuscation techniques are used. The results show transformer-based models are well-suited for intrusion detection to protect web applications from SQL injection, even when attackers actively attempt to evade detection.

Keywords: SQLi: SQL injection; NLP; text classification; adversarial robustness; BERT; intrusion detection; OWASP

1. Introduction

Even with years of research and available cybersecurity tools, SQLi remains one of the Top 10 OWASP issues and continues to be a major source of data breaches, costing companies and harming individuals' privacy.

SQL injection is a vulnerability in user-facing web forms or Application Programming Interfaces (API)s that require user or device input. The web form or API includes input fields or accepts HTTP POST requests that may not be validated and can be used to execute actual SQL queries. If the query is accepted, sensitive information from the database may be provided. In addition, the database location could be inadvertently exposed, allowing a malicious actor to download the entire database, which contains personal and sensitive information.

In modern applications, SQL queries are generated and handled across different systems. Backend services, APIs, and microservices routinely construct and execute queries as part of normal operation. This has been exacerbated with the explosion of cloud server services, such as Supabase, Amazon RDS, Azure MySQL, and Google Cloud. These cloud services allow web applications to operate globally, communicating with backend services using SQL across different networks.

Standard SQLi detection methods rely on signature-based rules to identify known attacks (Uwagbole et al., 2017). These approaches can identify specific patterns. However, if the pattern is altered, attackers can easily avoid detection. Simple changes, such as URL encoding, inserting comments, or altering letter casing, can hide malicious payloads.

Natural language processing provides an alternative approach to inspection (Devlin et al., 2019). SQL queries can be viewed as sequences of tokens with a structure similar to natural language. Models can be used to detect malicious queries. Instead of relying on fixed signatures, these models learn patterns from data and can identify new attacks that share similar characteristics to previously seen examples.

In this paper, we make three contributions. First, we compare four classification models that represent different approaches: classical machine learning-Logistic Regression with TF-IDF features, deep learning -(CNN) Convolutional Neural Network, and transformer-based models-DistilBERT, and BERT. The data used is labeled as normal SQL and SQL injection datasets for training, validation, and testing. We then evaluate how well each model performs under two evasion scenarios that reflect realistic attacker behavior. Finally, we examine how these obfuscation techniques affect different types of models and discuss what this

means for building more effective defensive systems against SQL injection.

Both BERT and DistilBERT were evaluated to understand how transformer-based encoders perform on SQL injection detection. These models analyze the full input sequence, which could provide detection even if queries are modified. DistilBERT was included as a more efficient variant, allowing a direct comparison between a smaller model and the full BERT architecture in terms of performance and computational cost.

2. Related Work

2.1 Machine Learning Approaches to SQL Injection Detection

There are many examples using machine learning to detect SQL injection attacks. For example, a study from the University of Edinburgh presented a framework that extracts features from tokenized SQL queries and splits them up into different cases (Uwagbole et al., 2017). Then it is trained using a Support Vector Machine (SVM) classifier. Their results show that these models can detect malicious queries when clear structural patterns are present. Machine learning approaches rely on feature engineering, which makes them less flexible when facing new or modified attacks. When queries are obfuscated through encoding or structural changes. Many of the feature engineering techniques become invalid. Therefore, performance tends to drop as they are no longer able to detect properly.

In this work, transformer and deep learning-based models are evaluated directly from raw SQL query sequences. The goal is to see whether these models can capture deeper patterns in queries, especially when obfuscation and encoding are introduced.

2.2 Transformer and Deep Learning Models in Security Contexts

The introduction of models such as BERT has changed how many NLP tasks are approached (Devlin et al., 2019). Rather than relying on handcrafted features, transformer-based models process the entire input sequence and learn relationships between tokens during training. This has led to strong performance on tasks like question answering and named entity recognition.

Before transformers became common, deep learning techniques were used for text classification. Convolution Neural Networks (CNN)s, a type of deep learning model, work well for identifying local patterns, but they are less effective at capturing relationships across longer inputs. In this project, SQL queries are relatively short, so this limitation is reduced. Still, it is important to evaluate how well a CNN performs when queries are obfuscated.

In cybersecurity, deep learning and transformer-based models have been used for problems specifically with phishing detection and spam filtering. However, their use in SQLi detection is less explored, particularly when adversarial techniques are involved to evade detection. This motivates the comparison in this study, which examines how transformer-based models compare with deep learning and other machine learning techniques.

2.3 Evasion and Adversarial Robustness

Attackers often use techniques such as URL encoding, inserted comments, or added whitespace to perform SQLi and bypass signature-based rules. These changes to SQL query format can break simple detection rules and also affect models trained on clean data.

To represent these techniques, obfuscation was applied to the malicious queries in this dataset. This allows for the evaluation of how each model behaves when the input is intentionally modified. Testing under these conditions is important, since real-world attacks rarely use clean or consistent formats.

3. Methods

3.1 Dataset Description

The dataset used is sourced primarily from publicly available SQL injection datasets on Kaggle, supplemented with modern payloads and real-world examples from OWASP. The dataset consists of SQL query strings labeled as either benign (Label = 0) or malicious (Label = 1). The data is then split into three dataframes: train, validate, and test. The training set contains 21,643 examples, while the validation and test sets each contain 4,638 examples. The class distribution is somewhat imbalanced across all splits. About 60 percent of the data is benign, and 40 percent is malicious. This same

distribution is consistent in the validation and test sets.

Looking at the queries themselves, the benign samples follow normal SQL structure. These include SELECT statements with proper WHERE clauses, JOIN operations, and ordered results.

The malicious queries show common SQL injection patterns. These patterns include conditions WHERE 1=1, UNION SELECT statements, time-based payloads using functions such as SLEEP(), and error-based techniques using functions such as EXTRACTVALUE, REGEXP_SUBSTRING, and String non-completion (‘). SQL query strings are supported across multiple databases, including PostgreSQL, MySQL, and Oracle.

3.2 Obfuscated Test Sets

To test how well the models handle adversarial input, two modified versions of the test set were created. In both cases, only the malicious queries were changed. The benign queries were left the same. This reflects what would happen in a real attack, since normal queries are not altered, but malicious ones are.

The first version applies several simple changes to the malicious queries. Each character is randomly changed to uppercase or lowercase. Extra spaces are added after tokens. SQL comments are also inserted after keywords such as SELECT, UNION, OR, AND, and WHERE. In total, 1,803 malicious queries were modified. The final dataset still contains the same 2,835 benign queries and 1,803 malicious ones, but the malicious queries now look different on the surface.

The second version applies URL encoding to the same 1,803 malicious queries, replacing characters with their hexadecimal representations. For example, spaces become %20, and quotes become %27. It represents a character in hexadecimal format from 0-9, A-F. The character will have two hexadecimal digits.

3.3 Logistic Regression with TF IDF

As a baseline, we trained a Logistic Regression classifier using TF-IDF features with unigram and bigram representations. The vocabulary was limited to 5,000 features. All text was converted to lowercase before feature extraction. The model was trained on the training set for up to 1,000 iterations without additional hyperparameter tuning.

This machine learning model provides a baseline to compare more complex models. In practice, the TF-IDF representation captures common tokens and short patterns in SQL queries, but it does not account for the overall structure of the query. Since Logistic Regression with TF-IDF uses a bag-of-words approach, it does not preserve sequence information. Therefore, it is limited in its ability to generalize. As a result, when obfuscation techniques such as encoding or character insertion are applied, many of these patterns are altered or lost. This makes it harder for the model to correctly identify malicious SQLi queries.

3.4 Convolutional Neural Network

We trained a CNN model on tokenized SQL queries. The queries were processed using the Keras Tokenizer with a vocabulary size of 10,000, and each query was padded to a maximum length of 100 tokens.

The model begins with a trainable embedding layer of size 100, followed by three convolutional layers with filter sizes of 3, 4, and 5, each using 32 filters. The outputs are combined using global max pooling, followed by a dropout layer with a rate of 0.5. A dense layer with 32 units is applied before the final sigmoid output for binary classification.

This architecture allows the model to capture local patterns in token sequences, such as short SQL fragments, while still maintaining some sensitivity to word order. The model was trained for 5 epochs using the Adam optimizer and binary cross-entropy loss. The CNN captures local patterns in the query but only within a limited context window, which can become an issue when the pattern changes (Kim, 2014).

3.5 DistilBERT

DistilBERT is a compressed version of BERT designed to be faster and more efficient, while retaining BERT’s performance (Sanh et al., 2019). In this project, DistilBERT-base-uncased was fine-tuned for binary classification using the Hugging Face Transformers library.

SQL queries were tokenized using the model’s tokenizer, with padding and truncation applied to a

maximum sequence length of 128 tokens. This length was chosen because most queries in the dataset are short, and longer sequences did not provide any additional benefit. The tokenizer uses a WordPiece approach, which breaks input into subword units. This allows the model to handle uncommon or modified tokens, which is particularly useful when SQL queries are obfuscated.

Training was performed with a batch size of 16 and a weight decay of 0.01 to help regularize the model and reduce overfitting. The model was trained for 3 epochs, which was sufficient for convergence based on validation performance. After each epoch, performance was evaluated on the validation set, and the checkpoint with the highest F1 score was selected for final evaluation.

DistilBERT was included in this study to evaluate whether a smaller, more efficient transformer model could maintain strong performance in SQL injection detection, even under obfuscation.

3.6 BERT

In addition to DistilBERT, the full BERT-base-uncased model was also fine-tuned on the same dataset to provide a direct comparison between a compressed model and a larger transformer architecture. BERT processes the entire input sequence and captures bidirectional context, which can help the model learn relationships of the full query even when tokens have been rearranged or modified.

The same preprocessing steps were applied, including tokenization with padding and truncation to a maximum length of 128 tokens. Due to limited memory and CPU, the batch size was reduced to 8. Aside from this adjustment, the training configuration remained consistent with DistilBERT. Like DistilBERT, BERT uses WordPiece tokenization, which helps preserve meaningful patterns even when tokens are altered or partially encoded.

The model was trained for 3 epochs, and performance was evaluated on the validation set after each epoch. The checkpoint with the highest F1 score was selected for final evaluation.

Including BERT allows us to assess whether the additional model capacity improves detection performance or whether a lighter model, such as DistilBERT, is sufficient for this task.

3.7 Model Performance Evaluation

Model performance was evaluated using accuracy, precision, recall, and F1 score. The F1 score is used as the primary metric because it balances precision and recall.

4. Results

4.1 Performance on Clean and Lexical Obfuscation Test Sets

All models achieved strong performance on the clean test dataset, with high accuracy and F1 scores. Logistic Regression with TF-IDF features and the CNN both perform well on standard SQL queries without obfuscation. DistilBERT and BERT reached near-perfect results, with high precision and recall across both classes. These results suggest that all models can learn SQL injection patterns when the input closely matches the training data. Under these conditions, even simpler models are effective at distinguishing between benign and malicious queries.

When combined lexical obfuscation techniques were applied, including random casing, inserted whitespace, and SQL comments, performance remained unchanged. Although the queries appeared different, the underlying tokens were still recognizable in the TF-IDF representation. Because all queries are lowercased before tokenization, casing changes had no effect. The CNN also maintained strong performance, with F1 scores near 0.99, indicating that it generalized well to minor variations in the input.

DistilBERT and BERT similarly showed no meaningful degradation, continuing to classify nearly all malicious queries correctly. Overall, these findings indicate that combined lexical obfuscation techniques, while altering the visual appearance of SQL queries, do not significantly affect model performance because they preserve much of the original token structure.

4.2 Performance Under URL Encoding

A significant performance difference appeared when URL encoding was applied to the malicious queries. The URL encoding of characters alters the input's structure by converting it to its hexadecimal

representation. Under this condition, Logistic Regression failed to detect malicious queries, with recall dropping to zero. This indicates that the model was unable to recognize any of the encoded attack patterns, as the learned vocabulary no longer matched the transformed input.

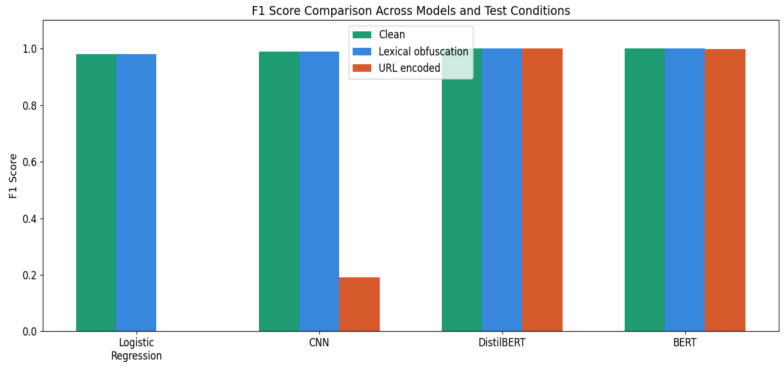
The CNN model also experienced substantial degradation. Many encoded tokens were mapped to unknown representations, limiting the model's ability to extract meaningful features. As a result, the detection rate dropped significantly, making the model unreliable under this type of obfuscation.

DistilBERT and BERT continued to perform exceptionally well, maintaining near-perfect recall and F1 scores. Their use of subword tokenization allowed them to break encoded sequences into smaller components, preserving enough structure for accurate classification. This demonstrates a key advantage of transformer-based models in handling adversarial input.

Table 1. Model Performance Across All Test Conditions

Model	Dataset	Acc.	Prec.	Rec.	F1
Logistic Reg.	Clean	0.99	0.99	0.97	0.98
Logistic Reg.	Lexical	0.99	0.99	0.97	0.98
Logistic Reg.	URL Encode	0.61	0.24	0.00	0.00
CNN	Clean	0.99	0.99	0.99	0.99
CNN	Lexical	0.99	0.99	0.99	0.99
CNN	URL Encode	0.65	0.89	0.11	0.19
DistilBERT	Clean	1.00	1.00	1.00	1.00
DistilBERT	Lexical	1.00	1.00	1.00	1.00
DistilBERT	URL Encode	1.00	1.00	1.00	1.00
BERT	Clean	1.00	1.00	1.00	1.00
BERT	Lexical	1.00	1.00	1.00	1.00
BERT	URL Encode	1.00	1.00	1.00	1.00

Figure 1. F1 Score Comparison Across Models and Test Conditions



4.3 Analysis of Model Robustness

The results highlight a clear difference in how models handle obfuscated inputs. Machine learning approaches such as Logistic Regression rely heavily on token-level representations derived from the training data. As a result, when queries are transformed using techniques such as URL encoding, important tokens become unrecognizable, leading to degraded performance. This is reflected in the recall dropping to zero, indicating that the model fails to identify any malicious queries under these conditions.

The CNN model demonstrates slightly better robustness but still suffers significant degradation. The CNN model depends on learned embeddings of tokenized inputs. Encoded sequences are often mapped to unknown tokens, limiting the model's ability to understand beyond the training distribution.

In contrast, transformer-based models such as BERT and DistilBERT maintain almost perfect performance across all scenarios. These models leverage subword tokenization and contextual attention mechanisms, allowing them to capture structural patterns even when queries are obfuscated or the format has been changed. As a result, they can generalize more effectively to unseen input variations.

Overall, these findings suggest that transformer-based approaches are significantly more robust to adversarial manipulation and better suited for real-world SQL injection detection, where attackers frequently attempt to evade detection through encoding and obfuscation.

5. Discussion

5.1 The Brittleness of Features Under Encoding

Traditional approaches based on lexical features, such as TF-IDF with Logistic Regression, depend heavily on tokens seen during training. When queries are transformed using techniques like URL encoding, those tokens no longer appear in their original form. As a result, the model cannot recognize patterns associated with malicious behavior. This leads to a complete breakdown in detection, with recall dropping to zero. In practice, this shows that purely lexical

approaches are extremely brittle, even against relatively simple encoding techniques.

5.2 Intermediate Robustness of CNNs

The CNN demonstrates partial resilience to combined obfuscation, but degrades significantly under URL encoding. Its recall on malicious examples falls to 11%, meaning that nearly 90% of obfuscated attacks would go undetected in a deployed system. This level of performance would be unacceptable in a real-world cybersecurity setting.

5.3 Transformer Robustness and Its Practical Implications

The sustained performance of both DistilBERT and BERT across all evaluation conditions is the most important result in this study. This appears to be driven by a combination of subword tokenization and contextual attention (Devlin et al., 2019). Instead of treating unseen inputs as completely unknown, subword tokenization breaks them into smaller, recognizable pieces. At the same time, the attention mechanism allows the model to capture relationships across the entire query. Together, these features allow transformer models to maintain high recall even when queries are heavily obfuscated. In practice, this means they are far more reliable at detecting real-world attacks. The results show identical recall on the obfuscated and clean test sets for SQLi queries.

5.4 Limitations

While the results are strong, there are a few limitations to consider. The dataset consists of labeled SQL queries and may not fully reflect real-world traffic. The obfuscation techniques used here were developed using a Python script to mimic similar attacks. However, attackers use a wide range of strategies that were not covered. All experiments were conducted offline. Performance considerations of the models inspecting live data, which could be affected by latency and changes in data over time, were not evaluated. Future work could explore more diverse datasets, additional adversarial techniques, and real-time deployment scenarios within operational systems and applications.

6. Conclusion

The report compared machine learning, deep learning, and transformer-based models for SQL injection detection under both clean and adversarial conditions. While all models performed well on clean test data, significant differences emerged when obfuscation techniques were introduced, specifically URL encoding, while lexical obfuscation methods did not cause any issues with the models, and malicious SQLi was able to be detected with a high recall.

Logistic Regression and CNN-based approaches showed substantial degradation, particularly under URL encoding, where recall dropped sharply. In contrast, transformer-based models such as BERT and DistilBERT maintained near-perfect performance across all scenarios. These results highlight the importance of model choice in security applications, where attackers actively attempt to evade detection. Approaches that rely on surface-level token patterns are vulnerable to simple transformations, while models that capture deeper structural relationships are more robust.

Overall, transformer-based models provide a more reliable foundation for detecting SQL injection attacks in real-world systems.

References

Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019).
BERT: Pre-training of deep bidirectional transformers for language understanding. *NAACL-HLT*.
https://arxiv.org/abs/1810.04805

Kim, Y. (2014).
Convolutional neural networks for sentence classification.
https://arxiv.org/abs/1408.5882

OWASP Foundation. (2021).
OWASP Top 10 Web Application Security Risks.
https://owasp.org/www-project-top-ten/

Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019).
DistilBERT: A distilled version of BERT.
https://arxiv.org/abs/1910.01108

Uwagbole, S. O., Buchanan, W. J., & Fan, L. (2017).
Applied machine learning predictive analytics to SQL injection attack detection.
Computer Networks, 115, 66–77.
https://doi.org/10.1016/j.comnet.2017.01.012